JSSMAHAVIDYAPEETHA



Mini Project/Internship Assessment

on

"CODET: An AI-Powered Code Quality Intelligence Tool"

Subject Name: Mini Project/Internship Assessment
Subject Code: BCS-752

COURSE: B.Tech.
SEMESTER: VII

by

Abhay Shukla (Roll No. _____)

Department of Computer Science and Engineering –AIML
JSS ACADEMY OF TECHNICAL EDUCATION C-20/1,
SECTOR-62, NOIDA

VISIONANDMISSION

VISION OF THE INSTITUTE

JSS Academy of **T**echnical **E**ducation Noida aims to become an Institution of excellence in imparting quality **O**utcome **B**ased **E**ducation that empowers the young generation with **K**nowledge, **S**kills, **R**esearch, **A**ptitude and **E**thical values to solve **Contemporary Challenging Problems.**

MISSION OF THE INSTITUTE

Develop a platform for achieving globally acceptable level of intellectual acumen and technological competence.

Create an inspiring ambience that raises the motivation level for conducting quality research.

Provide an environment for acquiring ethical values and positive attitude.

VISION OF THE DEPARTMENT

“To spark the imagination of the Computer Science Engineers with values, skills and creativity to solve the real-world problems.”

MISSION OF THE DEPARTMENT

To inculcate creative thinking and problem-solving skills through effective teaching, learning and research.
To empower professionals with core competency in the field of Computer Science and Engineering.
To foster independent and lifelong learning with ethical and social responsibilities.

PROGRAM OUTCOMES (POs)

Engineering Graduates will be able to:

PO1: Engineering knowledge: Apply the knowledge of mathematics, science, engineering fundamentals, and an engineering specialization to the solution of complex engineering problems.

PO2: Problem analysis: Identify, formulate, review research literature, and analyze complex engineering problems reaching substantiated conclusions using first principles of mathematics, natural sciences, and engineering sciences.

PO3: Design/development of solutions: Design solutions for complex engineering problems and design system components or processes that meet the specified needs with appropriate consideration for the public health and safety, and the cultural, societal, and environmental considerations.

PO4: Conduct investigations of complex problems: Use research-based knowledge and research methods including design of experiments, analysis and interpretation of data, and synthesis of the information to provide valid conclusions.

PO5: Modern tool usage: Create, select, and apply appropriate techniques, resources, and modern engineering and IT tools including prediction and modeling to complex engineering activities with an understanding of the limitations.

PO6: The engineer and society: Apply reasoning informed by the contextual knowledge to assess societal, health, safety, legal and cultural issues and the consequent responsibilities relevant to the professional engineering practice.

PO7: Environment and sustainability: Understand the impact of the professional engineering solutions in societal and environmental contexts, and demonstrate the knowledge of, and need for sustainable development.

PO8: Ethics: Apply ethical principles and commit to professional ethics and responsibilities and norms of the engineering practice.

PO9: Individual and teamwork: Function effectively as an individual, and as a member or leader in diverse teams, and in multidisciplinary settings.

PO10: Communication: Communicate effectively on complex engineering activities with the engineering community and with society at large, such as, being able to comprehend and write effective reports and design documentation, make effective presentations, and give and receive clear instructions.

PO11: Project management and finance: Demonstrate knowledge and understanding of the engineering and management principles and apply these to one’s own work, as a member and leader in a team, to manage projects and in multidisciplinary environments.

PO12: Life-long learning: Recognize the need for, and have the preparation and ability to engage in independent and life-long learning in the broadest context of technological change.

PROGRAM EDUCATIONAL OUTCOMES (PEOs)

PEO1: To apply computational skills necessary to analyze, formulate and solve engineering problems.

PEO2: To establish as entrepreneurs, and work in interdisciplinary research and development organizations as an individual or in a team.

PEO3: To inculcate ethical values and leadership qualities in students to have a successful career.

PEO4: To develop analytical thinking that helps them to comprehend and solve real-world problems and inherit the attitude of lifelong learning for pursuing higher education.

PROGRAM SPECIFIC OUTCOMES (PSOs)

PSO1: Acquiring in depth knowledge of theoretical foundations and issues in Computer Science to induce learning abilities for developing computational skills.

PSO2: Ability to analyse, design, develop, test and manage complex software system and applications using advanced tools and techniques.

COURSEOUTCOMES(COs)

C340.1	Developing a technical artifact requiring new technical skills and effectively Utilizing a new software tool to complete a task.
C340.2	Writing requirements documentation, Selecting appropriate technologies, Identifying and creating appropriate test cases for systems.
C340.3	Demonstratingunderstandingofprofessionalcustoms&practicesandworking With professional standards.
C340.4	Improving problem- solving, critical thinking skills and report writing.
C340.5	Learning professional skills like exercising leadership, behaving professionally, behaving ethically, listening effectively, participating as a member of a team, developing appropriate workplace attitudes.

CO-PO-PSOMAPPING

	PO1	**PO2**	**PO3**	**PO4**	**PO5**	**PO6**	**PO7**	**PO8**	**PO9**	**PO10**	**PO11**	**PO12**	**PSO1**	**PSO2**
C340.1	3	3	3	3	2	3	3	3	3	3	2	3	3	3
C340.2	3	3	3	3	3	3	3	3	3	2	3	3	3	3
C340.3	2	2	3	3	3	2	3	3	3	1	2	3	3	3
C340.4	2	2	2	2	2	2	2	2	2	3	2	3	2	2
C340.5	2	2	2	2	2	2	2	2	2	3	2	3	2	2
C340	2.40	2.40	2.60	2.60	2.40	2.40	2.60	2.60	2.60	2.40	2.20	3.00	2.60	2.60

DECLARATION

I hereby declare that this submission is my own work and that, to the best of my knowledge and belief, it contains no material previously published or written by another person nor material which to a substantial extent has been accepted for the award of any other degree or diploma of the university or other institute of higher learning, except where due acknowledgment has been made in the text.

Signature
: Name :
Roll No.:
Date :

CERTIFICATE

This is to certify that Mini Project/Internship Assessment Report entitled “CODET: An AI-Powered Code Quality Intelligence Tool” which

is submitted by Abhay Shukla in partial fulfillment of the requirement for the award of degree B. Tech. in Department of Computer Science and Engineering of Dr. APJ Abdul Kalam Technical University, Uttar Pradesh, Lucknow is a record of the candidate’s own work carried out by him/her under my supervision. The matter embodied in this report is original and has not been submitted for the award of any other degree.

Supervisor **Date**

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to the Department of Computer Science and Engineering (Artificial Intelligence and Machine Learning), JSS Academy of Technical Education, Noida, for giving me the opportunity to undertake this mini project as a part of the B.Tech. VII semester curriculum.

I am deeply thankful to my project supervisor for the constant guidance, constructive feedback and encouragement provided throughout the course of this work. Their suggestions on system design and on the evaluation of AI-generated results helped shape the project into its present form.

I extend my thanks to the Head of the Department and all faculty members of the department for providing the academic environment and laboratory facilities required for the successful completion of this project.

I also acknowledge the open-source community behind Python, FastAPI, LangChain, Qdrant, Redis, React and the many libraries on which this project is built. Their documentation and tooling made it possible to build a fairly complex system within the limited time frame of a semester.

Finally, I am grateful to my family and friends for their continuous support and motivation.

Abhay Shukla
B.Tech. CSE (AIML), VII Semester

ABSTRACT

Modern software projects grow quickly in size and complexity, and keeping their quality under control is a recurring challenge for developers and reviewers. Conventional static analysis tools such as linters and rule-based scanners are fast and precise for the patterns they know, but they cannot reason about intent, cross-file behaviour or project-specific conventions, and they offer little help when a developer simply wants to understand an unfamiliar codebase.

This report presents **Codet**, an AI-powered code quality intelligence tool that combines large language models (LLMs), retrieval-augmented generation (RAG) and a multi-agent architecture to analyse source code written in Python, JavaScript and TypeScript. An **Orchestrator Agent** receives a structural tree of the repository and strategically selects which files to inspect, while a **File Analysis Agent** examines each selected file and reports issues across eight categories (security, performance, duplication, complexity, testing, documentation, style and maintainability) with five severity levels, line numbers and concrete fix suggestions. The agents cooperate through a capability-scoped **shared memory** consisting of notes, todos with an explicit lifecycle, and a per-file analysis cache.

Large repositories are automatically parsed into semantic chunks using the Python AST and Tree-sitter, embedded with two sentence-transformer models (one for natural language and one for code) and stored in the Qdrant vector database, enabling hybrid semantic search during analysis and interactive question answering. Organisation-specific coding rules written in Markdown are indexed in the same way and retrieved per file. Conversation history and caches are kept in Redis.

The system is delivered through two interfaces: a rich command-line interface built with Click and Rich, and a web application consisting of a FastAPI backend with REST and WebSocket endpoints and a React dashboard that streams live agent progress, visualises the results and allows the user to chat with the codebase. The tool can run against Google Gemini in the cloud or a local model served by Ollama. The resulting reports include a prioritised issue list, category and severity breakdowns and a weighted quality score, giving developers an actionable and explainable view of their code quality.

Keywords: Code Quality, Static Analysis, Large Language Models, Multi-Agent Systems, Retrieval-Augmented Generation, Vector Database, FastAPI, React.

ABOUT THE INSTITUTE

JSS Academy of Technical Education (JSSATE), Noida was established in 1998 by the **JSS Mahavidyapeetha**, Mysuru, an educational trust that runs a large network of schools, colleges and universities across India and abroad. The institute is located at C-20/1, Sector-62, Noida, Uttar Pradesh, is approved by the All India Council for Technical Education (AICTE), New Delhi, and is affiliated to **Dr. A.P.J. Abdul Kalam Technical University (AKTU)**, Lucknow.

The institute offers undergraduate (B.Tech.) programmes in several engineering disciplines along with postgraduate programmes, and aims to impart quality outcome-based education that empowers students with knowledge, skills, research aptitude and ethical values, as reflected in its vision and mission statements presented at the beginning of this report.

The **Department of Computer Science and Engineering (Artificial Intelligence and Machine Learning)** offers a four-year B.Tech. programme that combines a strong foundation in computer science with specialised courses in machine learning, deep learning, natural language processing, data analytics and related areas. The department encourages students to apply these technologies to real-world problems through laboratory work, mini projects, internships and the final-year major project.

The mini project described in this report was carried out in the department as a part of the course **Mini Project/Internship Assessment (BCS-752)** in the VII semester. The project gave an opportunity to work with current AI technologies such as large language models, vector databases and agent frameworks, and to integrate them into a complete, usable software product.

TABLE OF CONTENTS

Title **Page No.**

Declaration.....vi
Certificate.....vii
Acknowledgement.....viii
Abstract.....ix
About the Institute.....x
List of Tables.....xiii
List of Figures.....xiv
List of Abbreviations.....xv
Chapter 1: Introduction.....1
1.1 Overview of the Project.....1
1.2 Project Background.....1
1.3 Problem Statement.....2
1.4 Objectives of the Project.....3
1.5 Scope of the Project.....3
1.6 My Role in the Project.....4
1.7 Organisation of the Report.....4
Chapter 2: Tools and Technology Used.....5
2.1 Python.....5
2.2 LangChain.....6
2.3 Large Language Models: Google Gemini and Ollama.....7
2.4 Qdrant Vector Database.....7
2.5 Sentence-Transformers Embedding Models.....8
2.6 Python AST and Tree-sitter.....8
2.7 Redis.....9
2.8 FastAPI and WebSockets.....9
2.9 React Frontend.....9
2.10 Click and Rich.....10
2.11 Supporting Tools.....10
2.12 Comparative Study and Justification.....10
Chapter 3: History and Features of the Technology.....12
3.1 History of Static Code Analysis.....12

3.2 History of Language Models for Code....12
3.3 Retrieval-Augmented Generation....13
3.4 Vector Databases....13
3.5 AI Agents and Multi-Agent Systems....14
3.6 Features of the Technology as Used in Codet....14
3.7 The Paradigm Shift....15
Chapter 4: Work Done....17
4.1 System Architecture....17
4.2 Use Case Diagram....18
4.3 Data Flow Diagrams....19
4.4 Overall Workflow....22
4.5 Module Information....23
4.6 Important Algorithms....28
4.7 Screenshots and Discussion....30
4.8 Testing and Validation....34
Chapter 5: Conclusions and Future Scope....36
5.1 Conclusion....36
5.2 Use of the Learned Technology in the Final Year Major Project....36
5.3 Industrial Relevance....37
5.4 Societal Relevance and Impact....37
5.5 Limitations....38
5.6 Future Scope....38
References....39
Appendices....41
Appendix A: Installation and Setup....41
Appendix B: Running the Tool....41
Appendix C: Sample Environment Configuration....41

LIST OF TABLES

Table No. and Title **Page No.**

Table 2.1 Technology stack of the project.....5
Table 2.2 Comparison of LLM options considered.....7
Table 2.3 Comparison of code analysis approaches.....10
Table 3.1 Key features of the core technologies.....14
Table 4.1 Backend modules and their responsibilities.....23
Table 4.2 Shared memory permissions by agent role.....25
Table 4.3 Tools available to the orchestrator agent.....26
Table 4.4 REST and WebSocket endpoints of the backend.....27
Table 4.5 Main React components.....28
Table 4.6 Representative test cases and expected behaviour.....35

LIST OF FIGURES

Figure No. and Title **Page No.**

Figure 4.1 System architecture of Codet....18
Figure 4.2 Use case diagram of Codet....19
Figure 4.3 Level 0 DFD (context diagram)....19
Figure 4.4 Level 1 DFD....20
Figure 4.5 Level 2 DFD for the file analysis process....21
Figure 4.6 Flowchart of the analysis workflow....22
Figure 4.7 Lifecycle of a todo item in shared memory....25
Figure 4.8 Landing page of the Codet web interface....30
Figure 4.9 Entering a GitHub repository URL for analysis....31
Figure 4.10 Client-side validation of the repository URL....31
Figure 4.11 Analysis categories presented on the landing page....32
Figure 4.12 Live processing status during an analysis....32
Figure 4.13 Analysis report displayed by the command-line interface....33
Figure 4.14 Analysis dashboard with metrics and issue cards....33
Figure 4.15 Interactive question answering about the analysed codebase....34

LIST OF ABBREVIATIONS

Abbreviation	Full Form
AI	Artificial Intelligence
AICTE	All India Council for Technical Education
AKTU	Dr. A.P.J. Abdul Kalam Technical University
API	Application Programming Interface
AST	Abstract Syntax Tree
CLI	Command Line Interface
CORS	Cross-Origin Resource Sharing
CSS	Cascading Style Sheets
DFD	Data Flow Diagram
HNSW	Hierarchical Navigable Small World (graph index)
HTTP	HyperText Transfer Protocol
JSON	JavaScript Object Notation
LLM	Large Language Model
MB	Megabyte
NLP	Natural Language Processing
OWASP	Open Worldwide Application Security Project
Q&A	Question and Answer
RAG	Retrieval-Augmented Generation
REST	Representational State Transfer
SAST	Static Application Security Testing
TTL	Time To Live
UI	User Interface
URL	Uniform Resource Locator
WS	WebSocket

CHAPTER 1
INTRODUCTION

1.1 Overview of the Project

Software quality is one of the most important factors that decides whether a project can be maintained, extended and trusted over time. As a codebase grows, defects, security weaknesses, duplicated logic and undocumented behaviour accumulate silently. Code reviews and automated linters help, but reviewers have limited time and linters only recognise the specific patterns they have been programmed for. A large part of the knowledge needed to judge code quality - what a function is supposed to do, whether a check is missing in some other file, whether a convention of the team is being followed - remains implicit.

Codet (short for *Code Quality Intelligence Tool*) is a mini project that attempts to close this gap by using large language models as reasoning engines over source code. Instead of matching fixed patterns, Codet reads the code the way a human reviewer would, guided by the structure of the repository, by semantic search over the whole codebase, and by organisation-specific rules supplied by the user. The output is a structured, prioritised quality report together with an interactive question-answering facility that lets a developer ask natural-language questions about the code.

The tool is implemented in Python on the backend and React on the frontend. It can be used either from the terminal through the `codet` command or through a web application in which a user can paste a GitHub repository URL or upload files and folders, follow the progress of the AI agents in real time, and explore the resulting report on a dashboard.

1.2 Project Background

Static analysis has a long history, from the original `lint` program for C to modern tools such as Pylint, ESLint, Bandit and SonarQube. These tools work by parsing source code and evaluating rules over the syntax tree or a control-flow representation. Their strengths are speed, determinism and a low false-negative rate for well-understood problems. Their weaknesses are equally well known: rules must be written by hand for each language and each problem, findings often lack context, the number of false positives can be high, and they cannot answer

questions such as *"where is the user authenticated and is that check applied to every API route?"*

In the last few years, transformer-based large language models trained on large corpora of source code and natural language have shown a strong ability to read, explain and modify programs. At the same time, techniques such as **retrieval-augmented generation (RAG)** and **tool calling** have made it practical to connect these models to external data and actions. This has led to a new generation of developer tools - AI pair programmers, automated reviewers and code-search assistants. Codet explores how these ideas can be combined into a focused tool for code quality assessment that is transparent about what it analysed and why.

1.3 Problem Statement

The problem addressed by this project can be stated as follows: *design and implement a system that accepts a source code repository (local or on GitHub), automatically identifies code quality issues across multiple categories with precise locations and actionable suggestions, scales to repositories that do not fit into the context window of a language model, respects project-specific coding rules, and allows developers to interactively ask questions about the codebase.*

Several sub-problems follow from this statement:

- **Context limits -** a language model can only read a limited number of tokens at once, so the system must decide which files are worth reading and must retrieve only the relevant parts of large codebases.
- **Coordination -** findings in one file often depend on another file (for example, a function that is never tested), so information discovered by one analysis step must be available to later steps.
- **Structured output** - the free-form text produced by a model must be converted into validated, machine-readable issues that can be counted, sorted and displayed.
- **Customisation** - every organisation has its own conventions, which must be injected into the analysis without retraining the model.
- **Usability** - long-running AI analyses must give feedback to the user while they run, through both a terminal and a browser.

1.4 Objectives of the Project

The main objectives of the project are:

1. To build a multi-agent analysis engine in which an orchestrator agent plans the analysis of a repository and delegates individual files to a specialised file analysis agent.
2. To detect and classify issues into eight categories (security, performance, duplication, complexity, testing, documentation, style and maintainability) with five severity levels, line numbers, code snippets and suggested fixes.
3. To implement automatic indexing of large repositories into a vector database using language-aware chunking (Python AST and Tree-sitter for JavaScript/TypeScript) and dual embeddings for hybrid search.
4. To support custom coding rules supplied as Markdown files, retrieved semantically for each analysed file.
5. To implement a shared memory through which agents can exchange notes, todos and cached results under role-based permissions.
6. To provide a command-line interface and a web interface (FastAPI + React) with live progress streaming, report visualisation, JSON export and an interactive chat over the analysed codebase.
7. To allow the use of either a cloud model (Google Gemini) or a locally hosted model (Ollama) so that private code need not leave the developer's machine.

1.5 Scope of the Project

The current version of Codet supports Python, JavaScript and TypeScript source files. It can analyse a single file, a local directory, an uploaded set of files or folders, or a public GitHub repository that is cloned on demand. Files and directories that are normally irrelevant to quality analysis - virtual environments, `node_modules`, build artefacts, caches, version-control metadata and anything listed in the repository's `.gitignore` - are filtered out automatically.

The tool is intended as an assistant to human reviewers rather than a replacement for them. Its findings are generated by a language model and must therefore be validated by the developer; for this reason every issue carries an explanation, a line reference and a suggested fix so that it can be checked quickly. Automatic modification of the source code is outside the scope of this version.

1.6 My Role in the Project

This project was designed and developed individually. My responsibilities covered the complete life cycle of the system:

- Studying existing static analysis tools and recent work on LLM-based code assistants, and defining the requirements of the system.
- Designing the overall architecture, including the orchestrator/file-analysis agent split, the shared memory model and the RAG pipeline.
- Implementing the backend in Python: agents and tool-calling loop, repository tree construction, file filtering, repository size checking, multi-language parsers, Qdrant indexers, Redis-based message history and the report generator.
- Implementing the command-line interface with live status display, and the FastAPI server with REST and WebSocket endpoints.
- Developing the React frontend: repository input and upload screen, live processing status, analysis dashboard, issue cards and chat panel.
- Testing the system, refining prompts and fixing issues found during code review.

1.7 Organisation of the Report

The rest of this report is organised as follows. **Chapter 2** describes the tools and technologies used in the project, discusses alternatives and justifies the choices made. **Chapter 3** presents the history and main features of the core technologies - static analysis, large language models, retrieval-augmented generation, vector databases and agent frameworks - and discusses the paradigm shift they represent. **Chapter 4** explains the work done: the architecture, use case diagram, data flow diagrams, module-wise description, algorithms and screenshots of the working system. **Chapter 5** concludes the report and outlines the industrial and societal relevance of the project and its future scope. References and appendices are given at the end.

CHAPTER 2
TOOLS AND TECHNOLOGY USED

Codet is a full-stack application that integrates several independent technologies. This chapter explains each of them, the role it plays in the project, the alternatives that were considered, and the reasons for the final choice. Table 2.1 summarises the complete technology stack.

Table 2.1: Technology stack of the project

Layer	Technology	Purpose in Codet
Language (backend)	Python 3.9+	Core implementation of agents, indexers, API and CLI
LLM framework	LangChain (core, google-genai, ollama)	Model abstraction, prompts, tool binding, structured output
Cloud LLM	Google Gemini (gemini-2.5-flash-lite)	Default reasoning model for analysis and chat
Local LLM	Ollama (llama3.2 by default)	Offline / private analysis
Vector database	Qdrant	Storage and search of code and rule embeddings
Embeddings	Sentence-Transformers (MiniLM-L6-v2, Jina code v2)	Natural-language and code vectors
Parsing	Python ast, Tree-sitter	Language-aware chunking of source code
Cache / history	Redis	Conversation history and response cache with TTL
Web backend	FastAPI, Uvicorn, WebSockets	REST API and live event streaming
Validation	Pydantic v2, pydantic-settings	Schemas for model output and configuration
CLI	Click, Rich	Command parsing and rich terminal output
Frontend	React 18, React Router, Axios, Chart.js	Web dashboard and chat interface
Package management	uv, pip, npm	Dependency and environment management
Version control	Git, GitHub	Source control and repository cloning

2.1 Python

Python was chosen as the primary implementation language because almost the entire modern AI ecosystem - model SDKs, LangChain, sentence-transformers, the Qdrant and Redis

clients - is available first and best supported in Python. Python also offers the built-in `ast` module, which gives direct access to the syntax tree of Python programs and is used by Codet's Python parser. Features such as `asyncio`, type hints and dataclasses are used extensively: agent calls are asynchronous so that the web server remains responsive while long analyses run, and data structures such as `CodeIssue`, `AnalysisResult`, `Note` and `Todo` are defined as dataclasses.

Alternatives: Node.js/TypeScript (good LLM SDK support, but weaker scientific and embedding libraries), Go or Java (better raw performance, but much less mature AI tooling). Since the bottleneck of the system is model latency rather than CPU time, Python's slower execution speed is not a limitation.

2.2 LangChain

LangChain is a framework for building applications on top of language models. Codet uses `langchain-core` for message types (`SystemMessage`, `HumanMessage`, `ToolMessage`), prompt templates and output parsers, and the provider packages `langchain-google-genai` and `langchain-ollama` to talk to Gemini and Ollama through a common chat-model interface. Because both providers implement the same interface, the rest of the code is independent of which model is used; switching to a local model is a single configuration flag (`use-local`).

LangChain's **tool binding** is used to expose Pydantic models (`AnalyzeFile`, `QueryFile`, `QueryCodebase`) as callable tools to the orchestrator. The model responds with structured tool calls, which Codet executes and feeds back as tool messages in a loop until the model produces a final answer. **Structured output** with Pydantic schemas is used to force the file analysis agent to return a JSON object containing a list of issues, notes and memory items.

Alternatives: calling the provider SDKs directly (less abstraction, but separate code paths per provider), LlamaIndex (strong for retrieval, less focused on custom agent loops), and newer agent frameworks such as LangGraph, CrewAI or AutoGen. LangChain was preferred because it offers provider portability and tool calling while still allowing a hand-written, transparent agent loop.

2.3 Large Language Models: Google Gemini and Ollama

The reasoning in Codet is performed by a large language model. The default is Google's **Gemini** family accessed through the Gemini API (the configured default is `gemini-2.5-flash-lite`, chosen for its low latency and cost, large context window and native support for tool calling). The model is used with a low temperature of 0.1 so that analyses are as repeatable as possible, and with a maximum output of 8192 tokens.

For users who cannot send their source code to a cloud service, Codet supports **Ollama**, a runtime that serves open-weight models such as Meta's Llama 3.2 on the local machine. Before starting an analysis in local mode, the CLI checks that the Ollama server is reachable on port 11434 and that the requested model has been pulled, and prints clear instructions otherwise.

Table 2.2: Comparison of LLM options considered

Criterion	Gemini (cloud)	Ollama (local)	OpenAI GPT (cloud)
Data privacy	Code sent to provider	Code stays on machine	Code sent to provider
Hardware needed	None	Capable CPU/GPU	None
Context window	Very large	Model dependent, smaller	Large
Tool calling	Native	Supported by recent models	Native
Cost	Low (free tier available)	Free after download	Paid per token
Quality on code	High	Moderate (model dependent)	High

2.4 Qdrant Vector Database

Qdrant is an open-source vector database written in Rust. It stores vectors together with a JSON payload and supports fast approximate nearest-neighbour search using HNSW graphs, payload filtering and **named vectors** (several vectors per point). Codet uses named vectors to store two embeddings for every code chunk - an `nlp` vector built from a natural-language description and a `code` vector built from the source itself - both compared with cosine distance. A second collection stores the chunks of the user's custom rule files. Qdrant can run locally in Docker, in memory for testing, or as a managed cloud service; Codet supports all three through configuration.

Alternatives: FAISS (a library, not a server - no persistence or filtering out of the box), Chroma (simple, but fewer production features), Pinecone (managed only, proprietary),

Weaviate and Milvus (feature-rich but heavier to operate), and PostgreSQL with pgvector. Qdrant was preferred for its named-vector support, payload filtering, simple Python client and free cloud tier.

2.5 Sentence-Transformers Embedding Models

Embeddings convert text or code into fixed-length vectors such that semantically similar inputs lie close together. Codet uses two models from the Sentence-Transformers library:

- **all-MiniLM-L6-v2** - a compact 384-dimensional general-purpose sentence embedding model, used for the natural-language description of each code chunk and for the rule chunks.
- **jina-embeddings-v2-base-code** - a 768-dimensional model trained specifically on source code in many programming languages, used for the code itself.

Using two models allows **hybrid search**: a natural-language question such as *"where are passwords hashed?"* is matched mainly against the descriptions, while a code snippet is matched against the code vectors. The results of both searches are merged and re-ranked. Running the models locally avoids additional API costs and keeps the indexed code private.

2.6 Python AST and Tree-sitter

To index a codebase meaningfully it must be split into units that make sense - functions, methods, classes and modules - rather than arbitrary blocks of lines. For Python, Codet uses the standard library `ast` module to walk the syntax tree and extract each function and class with its signature, docstring, decorators and exact line range. For JavaScript and TypeScript it uses **Tree-sitter**, an incremental parsing library that produces concrete syntax trees for many languages; Codet walks these trees to extract function declarations, arrow functions assigned to variables, classes and methods, together with their JSDoc comments.

Alternatives: regular-expression based splitting (fragile), fixed-size token windows (simple but breaks functions apart), or language servers (powerful but heavy). Tree-sitter was preferred because it is fast, robust to syntax errors, and supports many languages through a uniform API, which makes adding new languages straightforward.

2.7 Redis

Redis is an in-memory key-value store. Codet uses it to persist **conversation sessions** of the orchestrator agent (so that chat follow-up questions keep their context), as a **response cache** for repeated prompts, and to hold completed analysis results for the web dashboard. Message history entries expire after 24 hours and cache entries after one hour by default. If Redis is not available the agents fall back to an in-memory cache, so the tool still works in a minimal setup. **Alternatives** include SQLite or PostgreSQL (persistent but slower for this pattern) and Memcached (no rich data types).

2.8 FastAPI and WebSockets

The web backend is built with **FastAPI**, a modern asynchronous Python web framework based on Starlette and Pydantic, served by **Uvicorn**. FastAPI was chosen because it is natively asynchronous (important for long-running model calls), validates requests and responses automatically using the same Pydantic models used elsewhere in the project, and generates interactive OpenAPI documentation at `/docs`. **WebSocket** endpoints are used to stream agent events - tool start, tool completion, reasoning and memory updates - to the browser while an analysis or chat request is being processed.

Alternatives: Flask (synchronous by default, no built-in validation), Django (heavier, oriented to database-backed sites) and Node.js/Express (would split the backend across two languages).

2.9 React Frontend

The web interface is a single-page application written with **React 18** and bootstrapped with Create React App. **React Router** handles navigation between the analysis page and the dashboard, **Axios** performs HTTP requests, **Chart.js** (through `react-chartjs-2`) draws the category and severity charts, and **react-syntax-highlighter** renders code snippets inside issue cards. React was chosen for its component model, which maps naturally onto reusable UI elements such as issue cards, metric cards and status cards, and for its large ecosystem. Vue, Angular and Svelte were considered as alternatives.

2.10 Click and Rich

The command-line interface is implemented with **Click**, which provides declarative commands, arguments and options with automatic help text, and **Rich**, which renders colours, tables, panels, progress bars and live-updating displays in the terminal. Rich's `Live` display is used to show a continuously updated panel of agent events while the analysis runs.

2.11 Supporting Tools

- **Pydantic and pydantic-settings** - typed data validation; configuration is read from environment variables or a `.env` file into a `Settings` object.
- **gitignore-parser** - applies the repository's `.gitignore` rules when collecting files.
- **httpx** - HTTP client used, for example, to check the Ollama server.
- **Git** - used by the API to clone GitHub repositories into temporary directories.
- **uv** - a fast Python package and virtual-environment manager used for installation.

2.12 Comparative Study and Justification

Table 2.3 compares the approach adopted in Codet with the two most common alternative approaches to automated code quality analysis: traditional rule-based static analysers and simple single-prompt LLM usage (pasting code into a chatbot).

Table 2.3: Comparison of code analysis approaches

Aspect	Rule-based analysers (Pylint, ESLint, SonarQube)	Single-prompt LLM	Codet (agents + RAG)
Detection basis	Hand-written rules	Model reasoning	Model reasoning + retrieved context + custom rules
Cross-file reasoning	Limited	Only what is pasted	Shared memory and semantic search
Large repositories	Yes	No (context limit)	Yes (automatic indexing)
Custom conventions	Requires writing plugins	Must be repeated in prompt	Markdown rule files, retrieved per file
Explanations and fixes	Short messages	Yes, unstructured	Yes, structured with line numbers
Interactive Q&A	No	Yes, without codebase grounding	Yes, grounded in the codebase
Determinism	Fully deterministic	Low	Moderate (low temperature, schemas)

The comparison shows that Codet does not aim to replace deterministic linters, which remain the best choice for formatting and well-defined rule checks. Instead it complements them with capabilities that rules cannot provide: semantic understanding, cross-file reasoning, natural-language explanations and conversational exploration of a codebase. The chosen stack - Python, LangChain, Gemini/Ollama, Qdrant, Redis, FastAPI and React - was preferred because each component is open, well documented, replaceable, and available free of cost for academic use.

CHAPTER 3
HISTORY AND FEATURES OF THE TECHNOLOGY

Codet sits at the intersection of two lines of development: automated program analysis, which is several decades old, and large language models, which have matured only in the last few years. This chapter traces the history of both, describes the key features of the technologies used, and discusses the paradigm shift brought by AI-assisted code analysis.

3.1 History of Static Code Analysis

The idea of examining a program without running it is almost as old as programming itself. Compilers have always performed some checks, but the term *lint* comes from a tool written by Stephen C. Johnson at Bell Labs in 1978, which flagged suspicious constructs in C programs that the compiler accepted. Lint established the model followed by most later tools: parse the program, apply a set of heuristic rules, and report warnings.

During the 1990s and 2000s static analysis became more formal and more specialised. Tools based on data-flow analysis, abstract interpretation and model checking were developed for safety-critical software. Language-specific linters appeared for every popular language - for example Pylint for Python (2003), JSLint (2002) and later ESLint (2013) for JavaScript. Security-focused **static application security testing (SAST)** tools such as Bandit for Python emerged to detect injection flaws, unsafe deserialisation and hard-coded secrets, guided by lists such as the OWASP Top Ten.

Platforms such as **SonarQube** (2007 onwards) aggregated these checks into dashboards with metrics such as cyclomatic complexity, duplication percentage and technical debt, and integrated them into continuous integration pipelines. Metric libraries such as Radon compute McCabe complexity and maintainability indices for Python. These tools are still widely used today and form the baseline against which AI-based tools are compared.

3.2 History of Language Models for Code

Statistical language models for code were studied in the early 2010s, when researchers observed that source code is highly repetitive and predictable - the so-called *naturalness of software*. Early models were based on n-grams and later on recurrent neural networks, and were used for code completion and simple bug detection.

The decisive change came in 2017 with the **Transformer** architecture introduced in the paper *"Attention Is All You Need"*. Transformers process entire sequences in parallel using self-attention and scale well with data and compute. This led to pre-trained models such as BERT (2018) and the GPT series. Code-specific models followed: CodeBERT (2020), OpenAI Codex (2021), which powered the first version of GitHub Copilot, and later open models such as StarCoder and Code Llama (2023).

From 2022 onwards, instruction-tuned conversational models such as ChatGPT, Claude, Gemini and Llama demonstrated that a single general-purpose model could explain code, find bugs, write tests and review pull requests in natural language. Google's Gemini family (first released in December 2023) and Meta's openly available Llama models (Llama 3.2 in 2024) are the two families supported by Codet. Modern models also support **function calling** or **tool use**, in which the model returns a structured request to call an external function - the foundation of AI agents.

3.3 Retrieval-Augmented Generation

Language models know only what was in their training data and can read only a limited amount of text per request. **Retrieval-augmented generation**, introduced by Lewis et al. in 2020, addresses both limitations by retrieving relevant documents from an external store at query time and adding them to the prompt. RAG relies on dense vector embeddings - pioneered by models such as Sentence-BERT (2019) - and on efficient approximate nearest-neighbour search.

In Codet, RAG is used twice: to find relevant code anywhere in a large repository (via the `QueryCodebase` tool) and to find the custom coding rules that apply to the file currently being analysed. This lets the system work on repositories far larger than any context window while keeping each prompt small and focused.

3.4 Vector Databases

Similarity search over high-dimensional vectors was first studied for image and document retrieval. Libraries such as Facebook's FAISS (2017) made billion-scale search practical, and the HNSW graph algorithm (Malkov and Yashunin, 2016) became the most popular index structure. With the rise of RAG, dedicated **vector databases** - Pinecone,

Weaviate, Milvus, Chroma and Qdrant (first released in 2021) - added persistence, filtering, replication and APIs on top of these indexes.

The key features of Qdrant used in this project are:

- **Named vectors** - several embeddings per point, allowing separate natural-language and code vectors.
- **Payload storage and filtering** - each point stores the file path, chunk type, line range, signature and docstring, and searches can be restricted to a file or a chunk type.
- **Cosine similarity with HNSW indexing** - fast approximate search with high recall.
- **Flexible deployment** - in-memory mode for tests, a local server via Docker, or Qdrant Cloud.

3.5 AI Agents and Multi-Agent Systems

An **AI agent** is a program in which a language model repeatedly decides on an action, observes its result and decides again until a goal is reached. The ReAct pattern (reasoning and acting, 2022) and native tool calling in commercial models made this practical. Frameworks such as LangChain, LangGraph, AutoGen and CrewAI provide abstractions for agents, tools and memory.

Multi-agent systems divide work between agents with different roles and prompts. A common pattern, used in Codet, is an **orchestrator-worker** design: a planner agent sees the big picture and delegates focused sub-tasks to worker agents, each of which has a smaller, specialised context. Shared memory - inspired by the persistent notes and todo lists used by modern coding assistants - allows workers to leave observations and follow-up tasks for each other.

3.6 Features of the Technology as Used in Codet

Table 3.1: Key features of the core technologies

Technology	Key features	How Codet uses them
LLM (Gemini / Llama)	Code understanding, reasoning, tool calling, JSON output	Issue detection, planning, Q&A
LangChain	Provider-independent chat models, tools, parsers	Agent loop, structured output, model switching
RAG	Retrieve relevant context at query time	Codebase search, custom rule retrieval

Technology	Key features	How Codet uses them
Qdrant	Named vectors, filters, HNSW, cloud/local	Code and rule collections
Sentence-Transformers	Local dense embeddings	NL and code vectors for hybrid search
Tree-sitter / AST	Robust, fast syntax trees	Function/class level chunking
Redis	In-memory store with TTL	Chat history, response cache
FastAPI / WebSocket	Async, typed, live streaming	REST API, agent event streaming
React	Component-based reactive UI	Dashboard, issue cards, chat

3.7 The Paradigm Shift

Traditional static analysis answers the question *"does this code match a known bad pattern?"*. AI-assisted analysis answers a broader question: *"what would an experienced reviewer say about this code?"*. This shift has several consequences, summarised below.

- **From rules to reasoning** - instead of writing a new rule for every problem, the model generalises from its training to recognise problems it has never been explicitly told about, such as a missing authorisation check or a misleading function name.
- **From isolated warnings to explanations** - each finding comes with a description of why it matters, its likely impact, and a concrete fix, which lowers the effort needed to act on it.
- **From single files to whole codebases** - retrieval and shared memory let the analysis connect evidence across files, for example noticing that a function defined in one module is never tested.
- **From configuration files to natural-language rules** - team conventions can be written as plain Markdown and are applied semantically rather than syntactically.
- **From reports to conversations** - a developer can ask follow-up questions and explore the codebase interactively, which is especially valuable when onboarding onto an unfamiliar project.

The shift also introduces new challenges that Codet explicitly addresses. Model output is not deterministic, so Codet uses a low temperature and strict Pydantic schemas, and discards malformed results instead of guessing. Models can hallucinate, so every issue must point to a specific file and line and include the relevant snippet, making verification quick. Context windows are limited, so the orchestrator decides which files to read, large repositories are

indexed, and files larger than 1 MB or longer than 1000 lines are trimmed. Finally, privacy concerns are addressed by the option to run entirely on local models.

CHAPTER 4
WORK DONE

This chapter describes the design and implementation of Codet in detail. It begins with the overall architecture and the diagrams that model the system (use case diagram and data flow diagrams), then describes each module, the important algorithms, and finally presents screenshots of the working system with a discussion of their inputs, outputs and integration with other modules.

4.1 System Architecture

Codet follows a layered architecture, shown in Figure 4.1. The **presentation layer** consists of the command-line interface and the React web dashboard. The web dashboard talks to a **FastAPI backend** over HTTP and WebSockets, while the CLI calls the engine directly. Both use the **Orchestrator Engine**, which constructs the repository tree, initialises the agents, runs the analysis flow and compiles the final report.

The **agent layer** contains the Orchestrator Agent and the File Analysis Agent, both built on a common `BaseAgent` class that wraps the language model (Gemini or Ollama), prompt templates, caching and the tool-calling loop. The agents communicate through the **Shared Memory**. The **data and retrieval layer** contains the repository tree constructor and file filter, the codebase and rules indexers backed by Qdrant, and Redis for message history.

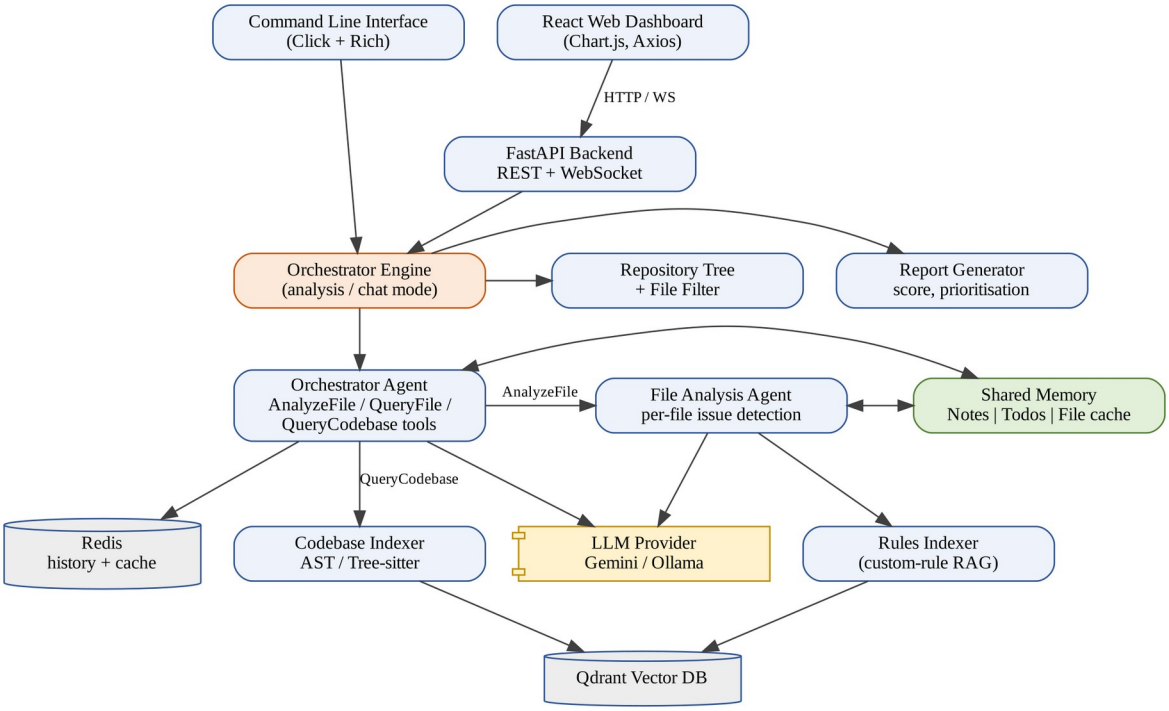


Figure 4.1: System architecture of Codet

4.2 Use Case Diagram

Figure 4.2 shows the use case diagram. The primary actor is the **developer**, who can analyse a local file or directory, analyse a GitHub repository, upload files or folders through the web interface, supply custom rule files, view the quality report and metrics, ask questions about the codebase, export the report as JSON, and follow the live progress of the agents. The **LLM service** is a secondary actor used by the analysis and chat use cases. Indexing of the codebase for RAG *extends* the analysis use case: it happens automatically for large repositories or when the user passes the `--index` flag.

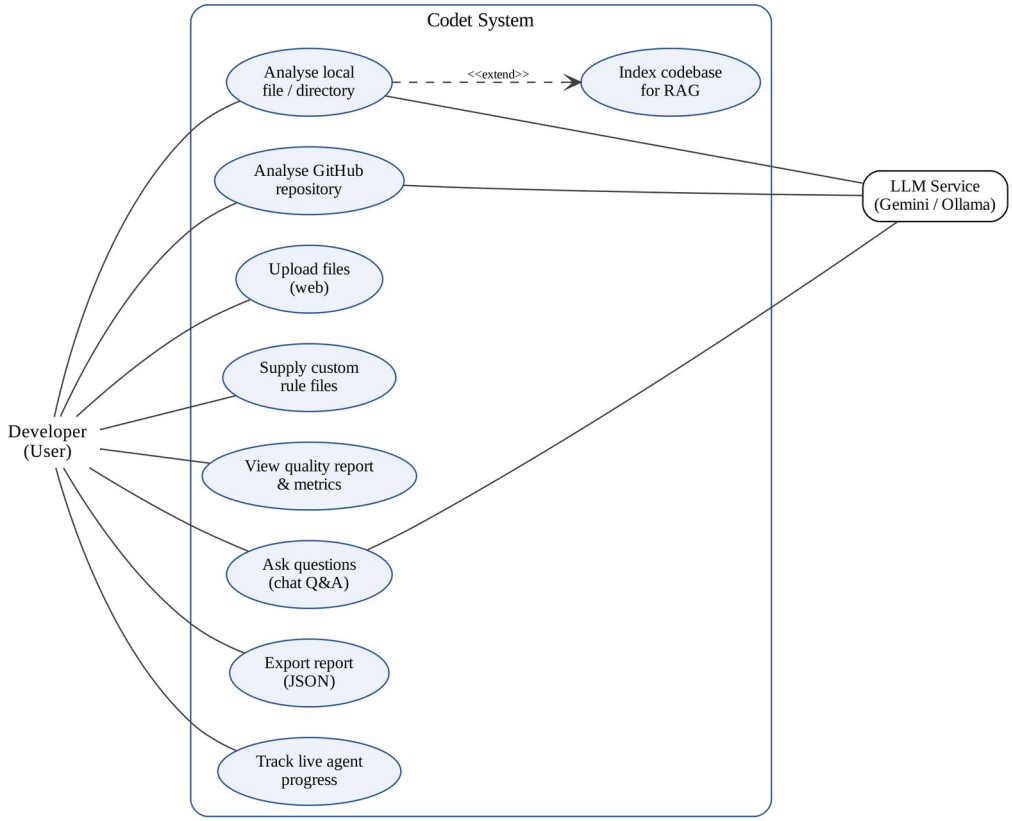


Figure 4.2: Use case diagram of Codet

4.3 Data Flow Diagrams

4.3.1 Level 0 DFD (Context Diagram)

The context diagram in Figure 4.3 treats the whole system as a single process. The developer supplies a source path or repository URL, optional rule files and questions; GitHub supplies repository files; and the LLM provider receives prompts with code context and returns structured issues, notes and answers. The system returns the quality report, metrics and answers to the developer.

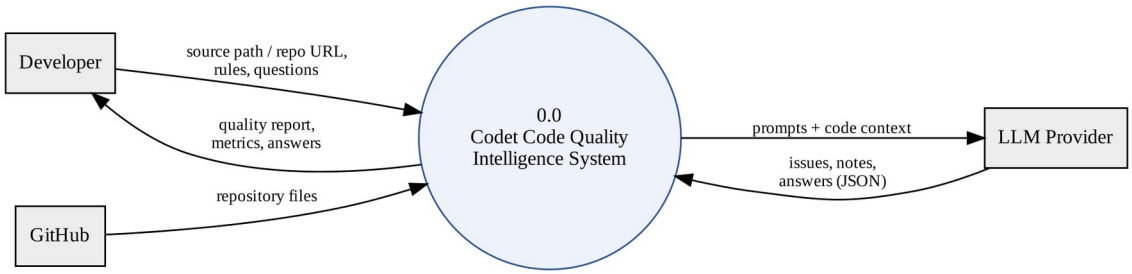


Figure 4.3: Level 0 DFD (context diagram)

4.3.2 Level 1 DFD

Figure 4.4 decomposes the system into seven processes: (1.0) acquiring the source by cloning, uploading or copying it to a temporary directory; (2.0) building the repository tree and checking the repository size; (3.0) indexing the codebase into Qdrant when required; (4.0) orchestrating the analysis; (5.0) analysing individual files with the LLM; (6.0) generating the report; and (7.0) answering follow-up questions. The data stores are the Qdrant vector collections (D1), the shared memory (D2) and the Redis message history (D3).

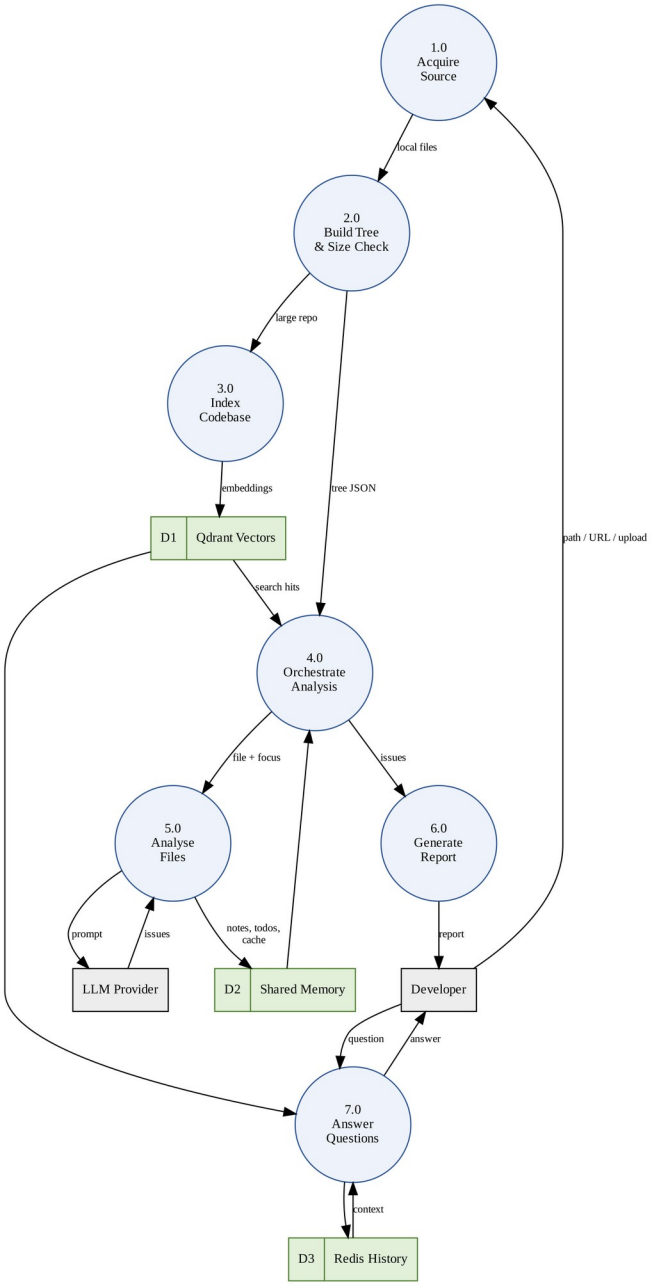


Figure 4.4: Level 1 DFD

4.3.3 Level 2 DFD (File Analysis Process)

Figure 4.5 expands process 5.0, the analysis of a single file. The file is read and trimmed to the configured limits (5.1); function and class names are extracted (5.2) and used to retrieve the most relevant custom rules from the rules collection (5.3); a prompt is assembled from the numbered code, the retrieved rules and the current shared-memory notes and todos (5.4); the LLM is invoked with a structured-output schema (5.5); the response is validated and converted into `CodeIssue` objects (5.6); and finally new notes, todos and the file's cached result are written to shared memory (5.7).

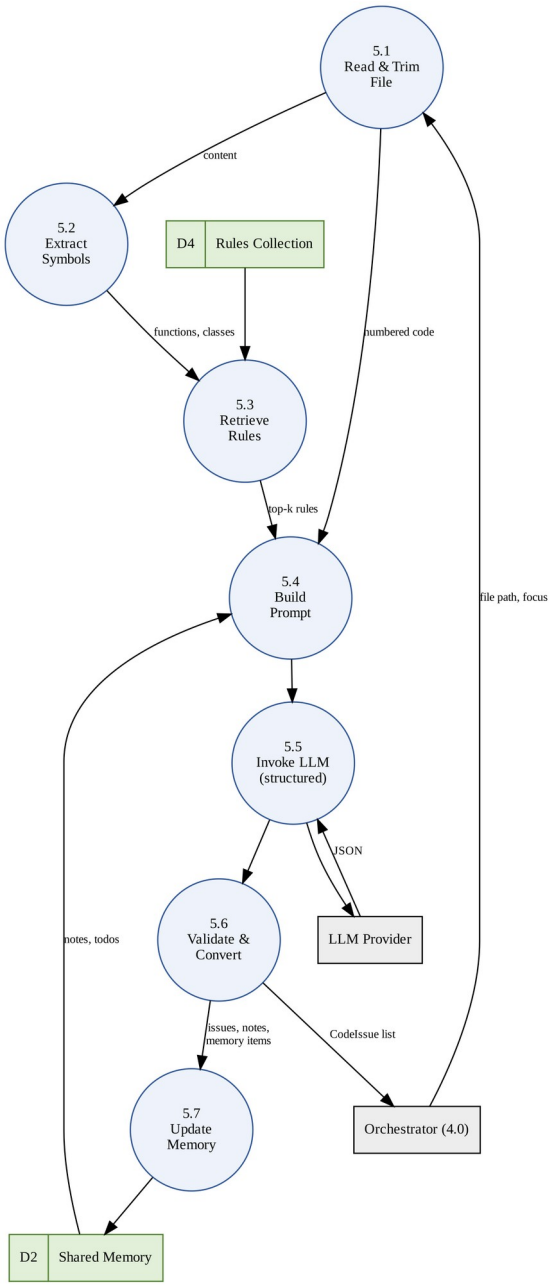


Figure 4.5: Level 2 DFD for the file analysis process

4.4 Overall Workflow

The flowchart in Figure 4.6 summarises the end-to-end analysis workflow. After the input is received and copied to a working directory, the repository size checker decides whether indexing is needed. The tree is then constructed and the orchestrator repeatedly selects a file, delegates it to the file analysis agent and updates shared memory, until it decides that no further tool calls are required. Finally the results are compiled into a report with a summary and quality score.

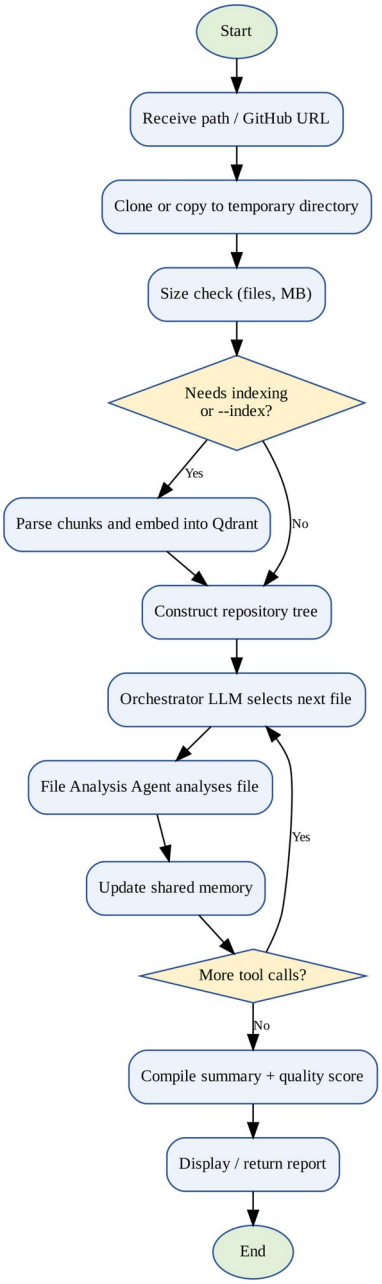


Figure 4.6: Flowchart of the analysis workflow

4.5 Module Information

The backend source code is organised into the packages listed in Table 4.1. Each module is described in the subsections that follow.

Table 4.1: Backend modules and their responsibilities

Package / file	Main classes	Responsibility
src/cli.py	analyze, serve commands	Command-line interface
src/core/orchestrator_engine.py	OrchestratorEngine	Runs the analysis / chat flow and compiles results
src/core/shared_memory.py	SharedMemory, MemoryView, Note, Todo	Cross-agent memory with role-based access
src/core/repository_tree.py	RepositoryTreeConstructor	Builds a JSON tree of the repository
src/core/config.py	Settings, Config, AgentConfig	Configuration from environment / .env
src/core/redis_client.py, message_history.py	RedisClient, MessageHistoryManager	Sessions, history and cache
src/agents/base_agent.py	BaseAgent	LLM wrapper, caching, tool-calling loop
src/agents/orchestrator_agent.py	OrchestratorAgent	Plans analysis, calls tools
src/agents/ file_analysis_agent.py	FileAnalysisAgent	Analyses a single file
src/agents/schemas.py, tools.py	CodeIssue, AnalysisResult, tool models	Data models and tool definitions
src/indexer/	MultiLanguageCodebaseParser, CodebaseIndexer, RulesIndexer	Parsing, embedding and retrieval
src/utils/	FileFilter, RepoSizeChecker, QdrantBase, CLIProcessingStatus	Shared utilities
src/reports/report_generator.py	ReportGenerator	Summary, quality score, prioritisation
src/api/main.py, models.py	FastAPI app	REST and WebSocket endpoints

4.5.1 Configuration Module

All settings are defined in a `Settings` class based on `pydantic-settings`, which reads values from environment variables or a `.env` file. Important settings include the Google API key and Gemini model, the agent temperature and token limit, the Ollama model, the Qdrant URL and API key, Redis connection parameters and TTLs, and the repository size thresholds used to decide when to index. A template `config/env.example` is provided. The

settings are converted into smaller typed objects (`AgentConfig`, `RedisConfig`) that are passed to the components that need them.

4.5.2 Repository Tree Constructor and File Filter

The `RepositoryTreeConstructor` walks the repository and produces a nested JSON structure describing directories and files, including file sizes and languages. This tree is the orchestrator's map of the project. The `FileFilter` class centralises the rules for what to skip: a default list of patterns (compiled Python files, `__pycache__`, `.git`, `node_modules`, `.env`, logs, build and distribution folders, virtual environments and coverage output), hidden files, and the rules of the repository's own `.gitignore` parsed with `gitignore-parser`.

4.5.3 Repository Size Checker

The `RepoSizeChecker` counts the supported source files and their total size, and recommends indexing when the repository contains at least **100** code files or at least **10 MB** of code (both configurable). Smaller repositories are analysed directly, which avoids the start-up cost of loading the embedding models. The user can force indexing with the `--index` flag.

4.5.4 Multi-Language Parser and Codebase Indexer

The `MultiLanguageCodebaseParser` dispatches each file to a language-specific parser. The `PythonASTParser` creates a module chunk from the module docstring and a chunk for every function, method and class, recording the signature (including argument types, defaults and return annotations), docstring, decorators, start and end lines and enclosing class. The `JavaScriptTreeSitterParser` does the same for JavaScript and TypeScript, including arrow functions assigned to variables and JSDoc comments. Each chunk is represented by the `CodeChunk` Pydantic model.

The `CodebaseIndexer` embeds each chunk twice - its natural-language description with MiniLM and its code with the Jina code model - and upserts it into a Qdrant collection with named vectors `nlp` and `code`, in batches of 32. It offers natural-language search, code search, **hybrid search** that merges and re-ranks both result lists, and search by chunk type or by file.

4.5.5 Rules Indexer

The `RulesIndexer` lets organisations supply their own coding guidelines as Markdown files (passed with `--rules`). It splits the Markdown into chunks by headings,

extracts metadata such as category, sub-category, language, priority, file patterns and keywords, embeds each chunk and stores it in a separate Qdrant collection. When a file is analysed, a query is built from the file path, the names of its functions and classes and whether it is a test file; twice the required number of candidates is retrieved and then re-ranked using keyword overlap and rule priority, as described in Section 4.6.3.

4.5.6 Shared Memory

The `SharedMemory` class is the single source of truth for information exchanged between agents. It has three layers:

- **Notes** - short, durable observations about the codebase (for example, *"authenticate() in src/auth.py is the single entry point for login"*), with optional tags and file path.
- **Todos** - action items with an explicit lifecycle: *pending*, *in_progress* and *done*, an optional owner and an optional target file (for example, *"check whether authenticate() has tests"*).
- **File cache** - the analysis result of each file, which prevents re-analysis and lets agents look up what has already been found.

Agents never modify the store directly. Each agent receives a `MemoryView`, a capability-scoped facade whose permissions depend on its role, as shown in Table 4.2. All mutations are thread-safe (guarded by a re-entrant lock), duplicate notes and todos are ignored, and every change emits a `memory_update` event that the CLI and the web interface display live. The todo lifecycle is illustrated in Figure 4.7.

Table 4.2: Shared memory permissions by agent role

Operation	Orchestrator	File analysis agent
Read notes, todos, file cache	Yes	Yes
Add notes and todos	Yes	Yes
Update todo status	Any todo	Own todos or todos targeting its file
Write file cache	Any file	Only the file it is analysing
Remove notes, prune todos, reset	Yes	No

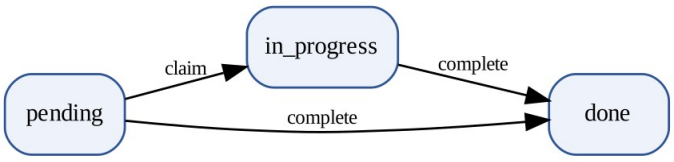


Figure 4.7: Lifecycle of a todo item in shared memory

4.5.7 Base Agent and Tool-Calling Loop

`BaseAgent` initialises either a `ChatGoogleGenerativeAI` or a `ChatOllama` model depending on the configuration, sets up prompt templates and an optional Redis connection, and provides three key methods: `generate_response` for plain text, `generate_structured_response` for Pydantic-validated JSON, and `run_tool_loop`, which implements the agent cycle *LLM -> tools -> LLM -> ... -> final response*. In each iteration the tools are bound to the model, the model's tool calls are executed through a handler supplied by the engine, the results are appended as tool messages, and the loop continues until the model replies without tool calls.

4.5.8 Orchestrator Agent

The orchestrator receives the repository tree and has access to three tools, defined as Pydantic models and listed in Table 4.3. Its system prompt instructs it to prioritise files referenced by pending todos, then entry points (such as `main.py`, `app.py` or `index.js`), core logic and configuration files, large or complex files, and finally tests and documentation. In **chat mode** the prompt instead asks it to analyse only the files relevant to the user's question and to answer with specific code details; previous analysis results and the Redis conversation history are available as context.

Table 4.3: Tools available to the orchestrator agent

Tool	Parameters	Action
AnalyzeFile	file_path, analysis_focus	Runs the file analysis agent on one file and returns its issues
QueryFile	file_path, question	Answers a specific question about one file
QueryCodebase	question, search_limit	Semantic search over the indexed codebase (only when indexed)

4.5.9 File Analysis Agent

The `FileAnalysisAgent` analyses one file at a time. Content beyond 1 MB is truncated and at most the first 1000 lines are analysed, with a note telling the model that the file was truncated. The code is presented with line numbers so that the model can report exact locations. Its system prompt lists the eight valid categories and asks for a clear description, line number, concrete suggestion and impact assessment for each issue. The response must conform to the `AnalysisResponseSchema`, which contains the list of issues plus `notes` and `memory_items` (new todos) for the shared memory. During conversion, unrecognised

categories default to *maintainability* and unrecognised severities to *medium*, and issues that cannot be tied to a file are discarded.

4.5.10 Report Generator

The `ReportGenerator` produces the summary of an analysis: total issues, counts by severity and by category, the number of files analysed, and a **quality score** between 0 and 100 (Section 4.6.2). It also sorts issues by priority so that critical security problems appear first.

4.5.11 API Module

The FastAPI application exposes the endpoints listed in Table 4.4. Uploaded files and cloned repositories are stored in temporary directories that are cleaned up afterwards. CORS is enabled for the frontend. Each analysis result is serialised and cached in Redis for one hour under an analysis identifier, which the dashboard uses to fetch the report and to open a chat session.

Table 4.4: REST and WebSocket endpoints of the backend

Method	Endpoint	Description
GET	/	Service information
GET	/api/health	Health check
POST	/api/analyze	Analyse a path on the server
POST	/api/upload	Upload files / folders for analysis
POST	/api/analyze-github	Clone and analyse a GitHub repository
POST	/api/ask/{analysis_id}	Ask a question about an analysed codebase
GET	/api/report/{analysis_id}	Fetch the report of an analysis
WS	/api/ws/analyze	Run an analysis with live event streaming
WS	/api/ws/chat/{analysis_id}	Chat with live event streaming

4.5.12 Command-Line Interface

The CLI is installed as the `codet` command. The `analyze` command accepts a file or directory path and the options shown below; the `serve` command starts the API server. While an analysis runs, a live Rich panel shows the most recent agent events - files being analysed, codebase queries, notes added and todos created or completed.

```
$ codet analyze PATH [OPTIONS]
  -o, --output PATH        Output file for the report
  -f, --format [console|json]
  -c, --config PATH        Configuration (.env) file
  --use-local              Use a local Ollama model instead of Gemini
  --ollama-model TEXT      Ollama model to use (default: llama3.2)
  --index                  Force indexing of the codebase for RAG
  --collection TEXT        Qdrant collection name
  --qdrant-url TEXT        Qdrant server URL
  --qdrant-api-key TEXT    Qdrant API key
  -r, --rules PATH         Custom rule markdown file (repeatable)

$ codet serve --host 0.0.0.0 --port 8000
```

4.5.13 Frontend Module

The React application consists of the components listed in Table 4.5. The `services/api.js` module wraps the REST calls, `utils/reportFormatter.js` converts the backend report into the shape required by the dashboard, and `utils/githubIssueHelper.js` builds pre-filled *New issue* links so that a finding can be turned into a GitHub issue with one click.

Table 4.5: Main React components

Component	Purpose
GitHubAnalyzer	Landing page: GitHub URL input, file/folder upload, starts analysis over WebSocket
ProcessingStatus	Live list of agent events and progress during analysis or chat
AnalysisDashboard	Report view: metrics, charts, issue list and chat panel
IssueCard	One issue with severity, category, location, snippet, suggestion and GitHub issue link
MetricCard / StatusCard	Summary numbers and status indicators
Header, Button, Input, Toast, LoadingSpinner	Reusable UI elements

4.6 Important Algorithms

4.6.1 Orchestrated Analysis Loop

The core analysis flow implemented by `OrchestratorEngine` and `OrchestratorAgent` can be summarised by the following algorithm:

1. Clear shared memory and construct the repository tree for the input path.

2. Build the analysis prompt containing the tree and the current memory summary, and register the tool handlers (`AnalyzeFile`, `QueryFile` and, if indexed, `QueryCodebase`).
3. Invoke the orchestrator model with the tools bound.
4. For each tool call returned: if it is `AnalyzeFile`, check the file cache; if not cached, run the file analysis agent with a file-scoped memory view, convert its issues, store them and emit events. For query tools, run the search or question and return the text result.
5. Append the tool results to the conversation and repeat from step 3 until the model returns no tool calls.
6. Compile all collected issues into an `AnalysisResult` with metrics, summary, timestamp and duration.

4.6.2 Quality Score Computation

Each issue is assigned a weight according to its severity: critical = 10, high = 5, medium = 2 and low/info = 1. If W is the sum of the weights of all n issues, the quality score is computed as:

```
score = max(0, 100 - (W / (10 * n)) * 100)       if n > 0
score = 100                                       if n = 0
```

The denominator $10n$ is the worst possible total (every issue critical), so the score reflects the average severity of the issues found: a codebase with only low-severity findings scores close to 90, while one dominated by critical issues approaches 0. Issues are then sorted by severity, then by category priority (security, performance, complexity, duplication, testing, maintainability, documentation, style), then by file path and line number.

4.6.3 Rule Retrieval and Re-ranking

For each analysed file, the rules indexer retrieves 2k candidate rule chunks (k = 5 by default) by cosine similarity. Each candidate's score is then adjusted: it is increased by 0.1 for every keyword of the rule that matches a function or class name in the file, and by a priority boost of +0.2 for critical rules, +0.1 for high, 0 for medium and -0.05 for low priority rules. The top k rules after re-ranking are inserted into the file analysis prompt.

4.6.4 Hybrid Code Search

When the orchestrator calls `QueryCodebase`, the question is embedded with the natural-language model and searched against the `nlp` vectors; if a code snippet is available it is also embedded with the code model and searched against the `code` vectors. The two result lists are merged by chunk identifier, combining scores for chunks found by both searches, and sorted by the combined score. The top results, with file paths, line ranges and signatures, are returned to the orchestrator.

4.7 Screenshots and Discussion

This section presents screenshots of the working system. For each screen, the feature of the technology used, its application in the project, its integration with other modules, and its inputs and outputs are discussed.

4.7.1 Web Interface: Landing Page

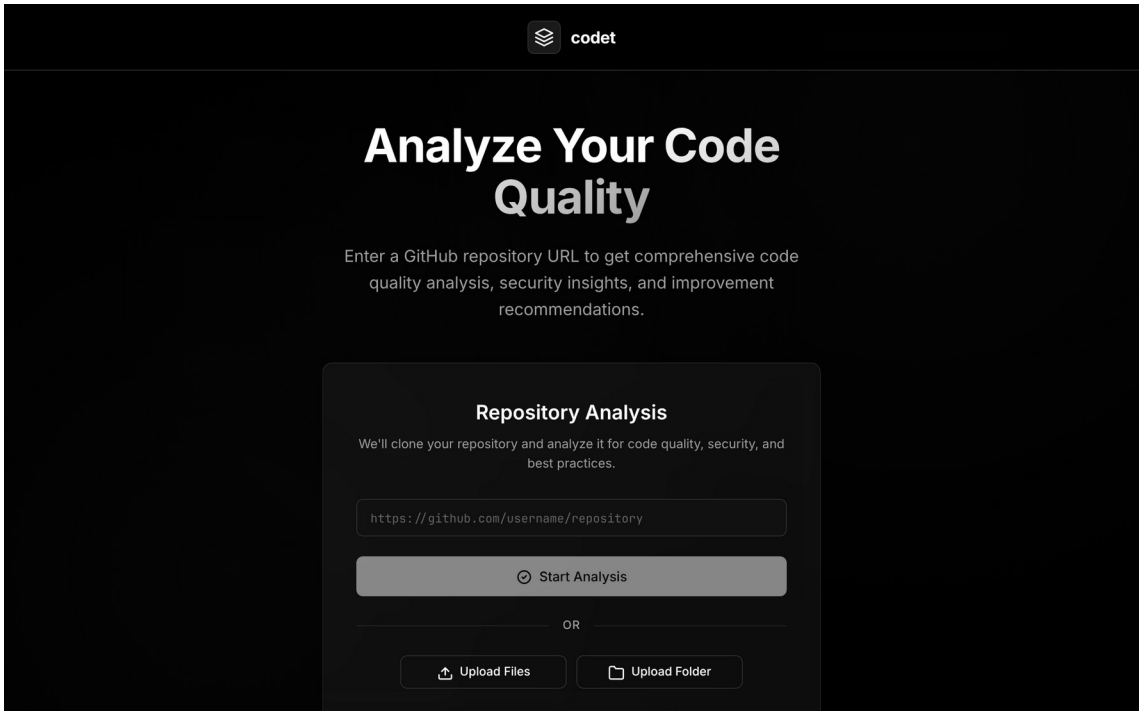


Figure 4.8: Landing page of the Codet web interface

Figure 4.8 shows the landing page rendered by the `GitHubAnalyzer` component. **Feature:** React components with state hooks render the page and manage the form state. **Application:** this is the entry point of the web workflow, where the user chooses what to analyse. **Integration:** on submission the component opens a WebSocket to `/api/ws/analyze` on the FastAPI backend, which in turn invokes the Orchestrator Engine. **Input:** a GitHub

repository URL, or files/folders selected with the upload buttons. **Output:** a new analysis session whose progress is displayed by the `ProcessingStatus` component and whose result opens in the dashboard.

4.7.2 Repository Input and Validation

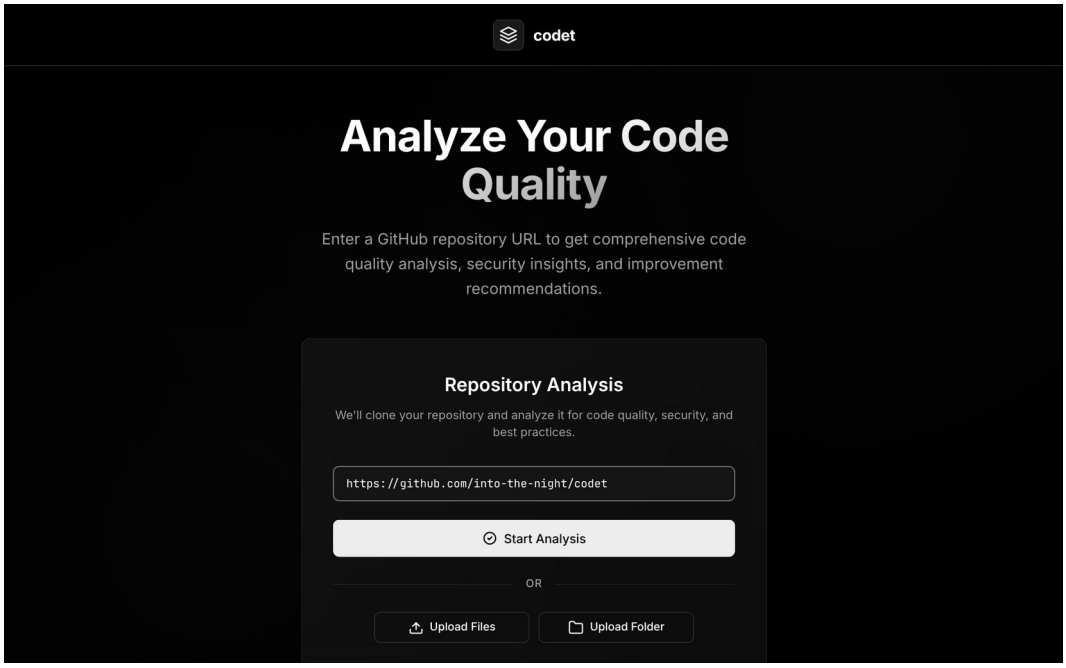


Figure 4.9: Entering a GitHub repository URL for analysis

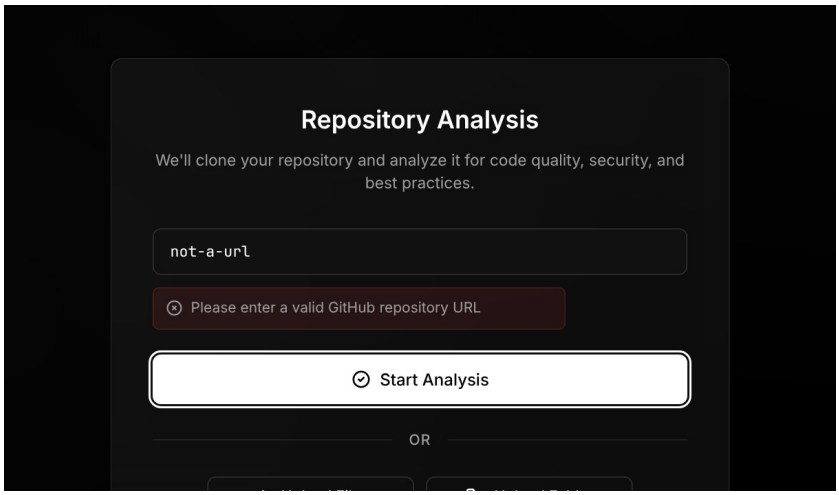


Figure 4.10: Client-side validation of the repository URL

Figure 4.9 shows a repository URL entered in the input field, and Figure 4.10 shows the validation message displayed when the input is not a valid GitHub URL. **Feature:** client-side validation with a regular expression gives immediate feedback without a round trip to the server. **Application:** it prevents invalid clone requests and wasted model calls. **Integration:** only validated URLs are sent to the backend, which clones the repository with Git into a

temporary directory prefixed `codet_github_`. **Input:** the URL text. **Output:** either an error message or the start of the analysis.

4.7.3 Analysis Categories

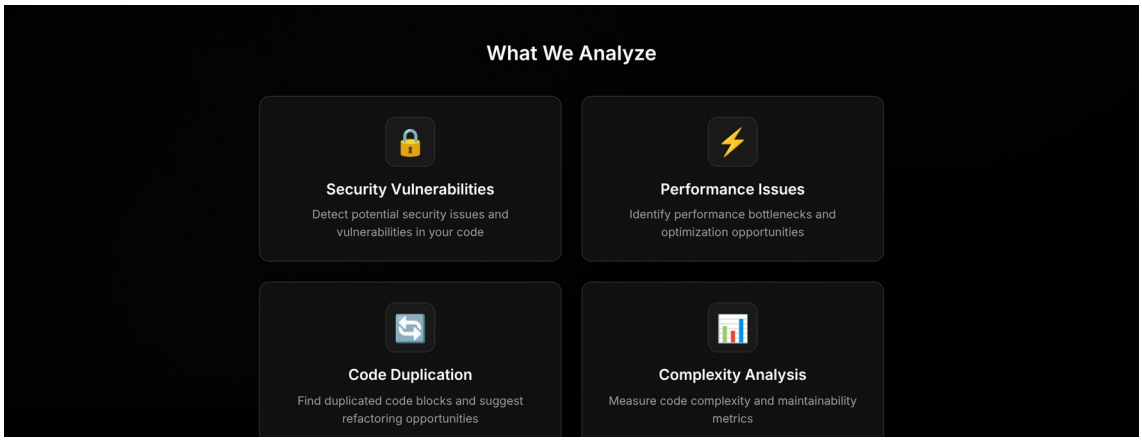


Figure 4.11: Analysis categories presented on the landing page

Figure 4.11 shows the *What We Analyze* section of the landing page, which presents the main categories of issues detected by the file analysis agent - security vulnerabilities, performance issues, code duplication and complexity. These correspond directly to the `IssueCategory` enumeration in the backend schemas, which additionally includes testing, documentation, style and maintainability.

4.7.4 Live Agent Progress

[Insert screenshot: run an analysis from the web interface (or the "codet analyze" command in the terminal) and capture the live panel showing agent events such as file analysis, codebase queries and memory updates.]

Figure 4.12: Live processing status during an analysis

Figure 4.12 shows the live status displayed while the agents work. **Feature:** WebSocket streaming in FastAPI on the web, and Rich `Live` panels in the terminal. **Application:** analyses can take from several seconds to minutes; live events make the process transparent by showing which files the orchestrator chose and why. **Integration:** the engine registers an event callback

with the agents and the shared memory; events of type `tool_start`, `tool_complete`, `reasoning` and `memory_update` are forwarded to the WebSocket or to the CLI status component. **Input:** agent events. **Output:** a continuously updated list of the most recent events.

4.7.5 Analysis Report in the Terminal

[Insert screenshot: output of the command "codet analyze <path>" showing the analysis configuration panel, the summary table (issues by severity and category, quality score) and the prioritised list of issues.]

Figure 4.13: Analysis report displayed by the command-line interface

Figure 4.13 shows the report produced by the CLI. **Feature:** Rich tables and panels format the summary and the issues with colour-coded severities. **Application:** developers can run quality checks from the terminal or in scripts, and export the result as JSON with `--format json --output report.json`. **Integration:** the CLI calls the Orchestrator Engine directly and uses the Report Generator for the summary. **Input:** the path and options. **Output:** the summary, quality score and a detailed, prioritised list of issues with file, line, description and suggestion.

4.7.6 Analysis Dashboard

[Insert screenshot: the web dashboard after an analysis completes, showing the metric cards, the severity/category charts and the list of issue cards.]

Figure 4.14: Analysis dashboard with metrics and issue cards

Figure 4.14 shows the analysis dashboard. **Feature:** React components with Chart.js charts and syntax-highlighted snippets. **Application:** the dashboard gives an overview of code quality at a glance and allows drilling down into individual issues; each issue card can create a pre-filled GitHub issue. **Integration:** the dashboard fetches the report from `/api/report/{analysis_id}` and formats it with `reportFormatter.js`. **Input:** the analysis report. **Output:** visual metrics and an interactive issue list.

4.7.7 Interactive Chat with the Codebase

[Insert screenshot: the chat panel of the dashboard with a question about the codebase and the answer produced by the orchestrator, including the live events shown while it was working.]

Figure 4.15: Interactive question answering about the analysed codebase

Figure 4.15 shows the chat panel. **Feature:** the orchestrator runs in chat mode with the same tools, the cached analysis result and Redis-backed conversation history. **Application:** developers can ask questions such as *"how is authentication handled?"* or *"which files have security issues?"* and receive answers that cite specific code. **Integration:** messages are sent over the `/api/ws/chat/{analysis_id}` WebSocket; for large repositories the `QueryCodebase` tool searches the Qdrant index. **Input:** a natural-language question. **Output:** a grounded answer with code references.

4.8 Testing and Validation

Validation of the system is planned at three levels. **Component testing** checks the deterministic parts - the file filter, repository size checker, parsers, chunk generation, report summary and quality score - against small repositories with known contents. **Integration testing** runs complete analyses through the CLI and the web interface in both Gemini and Ollama modes, with and without indexing and custom rules. **Robustness testing** covers empty directories, oversized files, invalid URLs, unavailable services and malformed model

responses. Table 4.6 lists representative test cases together with the behaviour the implementation provides for each.

Table 4.6: Representative test cases and expected behaviour

No.	Test case	Expected behaviour	Handled by
1	Analyse a single Python file	Issues reported with line numbers	CLI copies file to a temp directory
2	Directory containing node_modules and a .gitignore	Ignored paths are skipped	FileFilter
3	Repository with 100 or more code files	Indexing triggered automatically	RepoSizeChecker
4	Run with --use-local while Ollama is stopped	Clear error message and instructions	check_ollama_running()
5	Supply a custom rules Markdown file	Relevant rules retrieved per file	RulesIndexer
6	Invalid GitHub URL in web interface	Validation error shown	GitHubAnalyzer
7	Redis unavailable	Fallback to in-memory cache	BaseAgent
8	Model returns an issue with unknown category	Mapped to a default category	OrchestratorAgent
9	File longer than 1000 lines	Truncated with a note to the model	FileAnalysisAgent
10	Ask a follow-up question in chat	Answer uses previous context	Redis message history

CHAPTER 5
CONCLUSIONS AND FUTURE SCOPE

5.1 Conclusion

This project designed and implemented **Codet**, an AI-powered code quality intelligence tool that analyses Python, JavaScript and TypeScript codebases using large language models. The project demonstrates that an orchestrator-worker multi-agent design, combined with a shared memory of notes, todos and cached results, allows a language model to analyse a repository strategically rather than file by file in isolation. Retrieval-augmented generation over a Qdrant vector database, with language-aware chunking and dual embeddings, extends the approach to repositories that are far larger than a model's context window, and the same mechanism makes it possible to apply organisation-specific coding rules written in plain Markdown.

The system produces structured, prioritised reports with categories, severities, line numbers, code snippets, suggestions and a weighted quality score, and it supports interactive, codebase-grounded question answering. It is usable both from the terminal and from a web dashboard with live progress streaming, and it can run against a cloud model or entirely on a local model for privacy. All the objectives stated in Chapter 1 were achieved.

Working on this project provided hands-on experience with a wide range of current technologies: prompt design and structured output, tool calling and agent loops, embeddings and vector search, asynchronous web services, WebSockets, and modern frontend development. It also highlighted the practical challenges of building on language models - non-determinism, hallucination, context limits and latency - and the engineering techniques required to manage them.

5.2 Use of the Learned Technology in the Final Year Major Project

The technologies learned in this mini project form a strong foundation for the final-year major project. The agent framework, RAG pipeline and vector database skills are directly applicable to any system that must reason over a large private knowledge base - for example a domain-specific assistant for documents, research papers or enterprise data. The experience of building a FastAPI backend with WebSocket streaming and a React frontend can be reused for

the user-facing part of the major project. Codet itself can also be extended into the major project by adding automatic fix generation, pull-request integration and quantitative evaluation against benchmark datasets.

5.3 Industrial Relevance

- **Code review automation** - software companies spend a significant share of engineering time on code review. A tool that pre-screens changes and highlights likely problems lets reviewers focus on design and business logic.
- **Security** - early detection of injection flaws, hard-coded secrets and missing validation reduces the cost of fixing vulnerabilities, which is far higher after deployment.
- **Onboarding and knowledge transfer** - the chat feature helps new team members understand unfamiliar codebases quickly.
- **Enforcing internal standards** - custom rule files let organisations apply their own guidelines consistently without writing linter plugins.
- **Privacy-sensitive environments** - support for local models makes the approach usable in industries where code may not leave the premises.

5.4 Societal Relevance and Impact

Software is now part of almost every aspect of daily life - banking, healthcare, education, transport and government services. Defects and security vulnerabilities in this software can cause financial loss, privacy breaches and even physical harm. By making high-quality code review more accessible, tools such as Codet can contribute to safer and more reliable software.

The project also has an educational impact. Students, open-source contributors and small teams often do not have access to experienced reviewers. Codet's explanations and suggestions act as a mentor that explains *why* a piece of code is problematic and *how* it can be improved, helping developers learn good practices. Because the tool can run with free cloud tiers or entirely on local open-weight models, it is affordable for individuals and institutions with limited resources. At the same time, the project acknowledges that AI findings must be verified by humans, and it is designed to support rather than replace human judgement.

5.5 Limitations

- Findings are generated by a language model and may occasionally include false positives or miss issues; they must be reviewed by a developer.
- Only Python, JavaScript and TypeScript are currently supported by the parsers.
- Very large files are trimmed to the first 1000 lines, so issues beyond that point may be missed.
- Analysis quality and speed depend on the chosen model; small local models are less accurate.
- Analysis results in the web backend are cached in Redis for one hour only; there is no long-term report history.

5.6 Future Scope

1. **More languages** - adding Tree-sitter grammars for Java, Go, C/C++, Rust and others.
2. **Automatic fixes** - generating patches for selected issues and opening pull requests.
3. **CI/CD and pull-request integration** - a GitHub App or Action that analyses only the changed files of each pull request and posts review comments.
4. **Hybrid analysis** - combining deterministic tools such as Bandit, Pylint and Radon with the LLM, using their output as evidence and the LLM to explain, de-duplicate and prioritise.
5. **Persistent storage and history** - storing reports in a database to track quality trends over time.
6. **Evaluation** - measuring precision and recall on benchmark datasets of known vulnerabilities and bugs.
7. **IDE extensions** - bringing the analysis and chat directly into editors such as VS Code.
8. **Parallel file analysis** - analysing independent files concurrently to reduce total analysis time.

REFERENCES

[1] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser and I. Polosukhin, "Attention Is All You Need," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.

[2] P. Lewis et al., "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.

[3] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks," in *Proceedings of EMNLP-IJCNLP*, 2019.

[4] Y. A. Malkov and D. A. Yashunin, "Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 42, no. 4, pp. 824-836, 2020.

[5] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan and Y. Cao, "ReAct: Synergizing Reasoning and Acting in Language Models," in *International Conference on Learning Representations (ICLR)*, 2023.

[6] M. Chen et al., "Evaluating Large Language Models Trained on Code," arXiv:2107.03374, 2021.

[7] Z. Feng et al., "CodeBERT: A Pre-Trained Model for Programming and Natural Languages," in *Findings of EMNLP*, 2020.

[8] A. Hindle, E. T. Barr, Z. Su, M. Gabel and P. Devanbu, "On the Naturalness of Software," in *Proceedings of the 34th International Conference on Software Engineering (ICSE)*, 2012.

[9] S. C. Johnson, "Lint, a C Program Checker," Bell Laboratories, Computing Science Technical Report 65, 1978.

[10] M. Günther et al., "Jina Embeddings 2: 8192-Token General-Purpose Text Embeddings for Long Documents," arXiv:2310.19923, 2023.

[11] LangChain Documentation. [Online]. Available: https://python.langchain.com/

[12] Google AI for Developers, Gemini API Documentation. [Online]. Available: https://ai.google.dev/

[13] Qdrant Vector Database Documentation. [Online]. Available: https://qdrant.tech/documentation/

[14] Tree-sitter Documentation. [Online]. Available: https://tree-sitter.github.io/tree-sitter/

[15] FastAPI Documentation. [Online]. Available: https://fastapi.tiangolo.com/

[16] Redis Documentation. [Online]. Available: https://redis.io/docs/

[17] React Documentation. [Online]. Available: https://react.dev/

[18] Ollama Documentation. [Online]. Available: https://ollama.com/

[19] OWASP Top Ten. [Online]. Available: https://owasp.org/www-project-top-ten/

[20] Codet source code repository. [Online]. Available: https://github.com/into-the-night/codet

APPENDICES

Appendix A: Installation and Setup

```
# Clone the repository and prepare the environment
git clone https://github.com/into-the-night/codet.git
cd codet
cp config/env.example .env    # then add GOOGLE_API_KEY etc.

# Backend and CLI
uv venv && source .venv/bin/activate
uv pip install -r requirements.txt
uv pip install -e .

# Frontend
cd frontend && npm install && cd ..
```

Appendix B: Running the Tool

```
# Analyse a directory with Gemini
codet analyze /path/to/project

# Analyse with a local model and custom rules, export JSON
codet analyze ./src --use-local --ollama-model llama3.2 \
      --rules team_rules.md --format json --output report.json

# Start the web application
uvicorn src.api.main:app --reload       # backend on port 8000
cd frontend && npm start                # frontend on port 3000
```

Appendix C: Sample Environment Configuration

```
GOOGLE_API_KEY=your_google_api_key_here
REDIS_HOST=localhost
REDIS_PORT=6379
QDRANT_HOST=localhost
QDRANT_PORT=6333
QDRANT_COLLECTION=codet_embeddings
USE_LOCAL_LLM=false
OLLAMA_MODEL=llama3.2
OLLAMA_BASE_URL=http://localhost:11434
```